

Домашнее задание 8. Мониторинг

1. Определение ключевых бизнес- и технических метрик (Step_1.ipynb)

Проектирование системы метрик и ADR

В рамках разработки системы **Virtual Product Placement** спроектировано сбалансированное дерево метрик и подготовлено архитектурное обоснование (ADR) для разрешения конфликтов между командами бизнеса, разработки и эксплуатации.

1. Бизнес-уровень (Продукт и доход)

- **Выручка (LTV)**: Финальный финансовый показатель окупаемости системы.
- **CTR (Click-Through Rate)**: Отношение кликов к показам. Оценка интереса зрителя к вставленному бренду.
- **User Retention**: Мониторинг оттока аудитории. Контроль того, не вызывает ли автоматическая реклама раздражение у пользователей.

2. Технический уровень (IT-инфраструктура)

- **Throughput (10000 RPS)**: Пропускная способность. Соответствует требованию бизнеса по пиковой нагрузке.
- **Latency p95 < 200 мс**: Время обработки кадра. Гарантия плавности видеопотока без видимых задержек.
- **Error Rate < 1%**: Доля технических сбоев. Позволяет контролировать общую стабильность системы под нагрузкой.

3. ML-уровень (Качество алгоритмов)

- **Precision (Точность)**: Доля корректно определенных зон для вставки. Исключает репутационные риски (наложение бренда на лица).
- **IoU (Intersection over Union)**: Точность наложения графики. Определяет визуальный реализм и качество "склейки" объекта с кадром.
- **PSI (Population Stability Index)**: Мониторинг дрейфа данных. Сигнал о необходимости переобучения модели при изменении входного видеопотока.

4. Метрики инфраструктуры

- **CPU utilization (%)**: Загрузка процессора.
- **RAM usage (GB)**: Потребление оперативной памяти.
- **GPU utilization (%)**: Загрузка графического ускорителя.
- **Network I/O (Mbps)**: Входящий/исходящий трафик.

ADR: Переход на Карра-архитектуру

Контекст

Бизнес требует производительность 10 000 RPS. Текущая реализация на PostgreSQL (Docker-in-Docker) даёт не более 1000 RPS. ML-команде нужна потоковая обработка видео.

Решение

Внедрение **Карра-архитектуры**:

- Замена PostgreSQL на **ClickHouse** (аналитическая БД высокой производительности).
- Использование **Kafka** в качестве основной шины данных.

Причины

1. **Масштабируемость**: Kafka распределяет нагрузку, позволяет достичь 10 000 RPS.
2. **Производительность записи**: ClickHouse оптимизирован для высокой интенсивности вставок.
3. **Стриминг**: Карра-архитектура нативно поддерживает потоковую обработку.

Альтернативы (отвергнутые)

- **Lambda-архитектура** – сложна в поддержке (две кодовые базы).
- **PostgreSQL + шардирование** – не решает проблему аналитической производительности.

Риски

1. Повышение сложности инфраструктуры (требуется Kubernetes).
2. Высокое потребление RAM брокерами и ClickHouse.
3. Риск потери консистентности на пике (eventual consistency).

Компромиссы

- Сложность настройки -> выигрыш в производительности и горизонтальном масштабировании.
- Отказ от строгой согласованности (допустим для рекламных вставок).

```

from diagrams import Diagram, Cluster, Edge
from diagrams.onprem.monitoring import Prometheus
from diagrams.onprem.client import User
from diagrams.onprem.compute import Server
from diagrams.generic.compute import Rack

with Diagram("Дерево метрик ML-системы (4 уровня)", show=False, direction="BT")

    with Cluster("4. Инфраструктура (ресурсы)":
        infra = [
            Rack("CPU usage %"),
            Rack("RAM usage GB"),
            Rack("GPU usage %"),
            Rack("Network I/O Mbps")
        ]

    with Cluster("3. ML-уровень (качество модели)":
        ml = [
            Prometheus("Precision (точность зон)"),
            Prometheus("IoU (качество наложения)"),
            Prometheus("PSI (дрифт данных)")
        ]

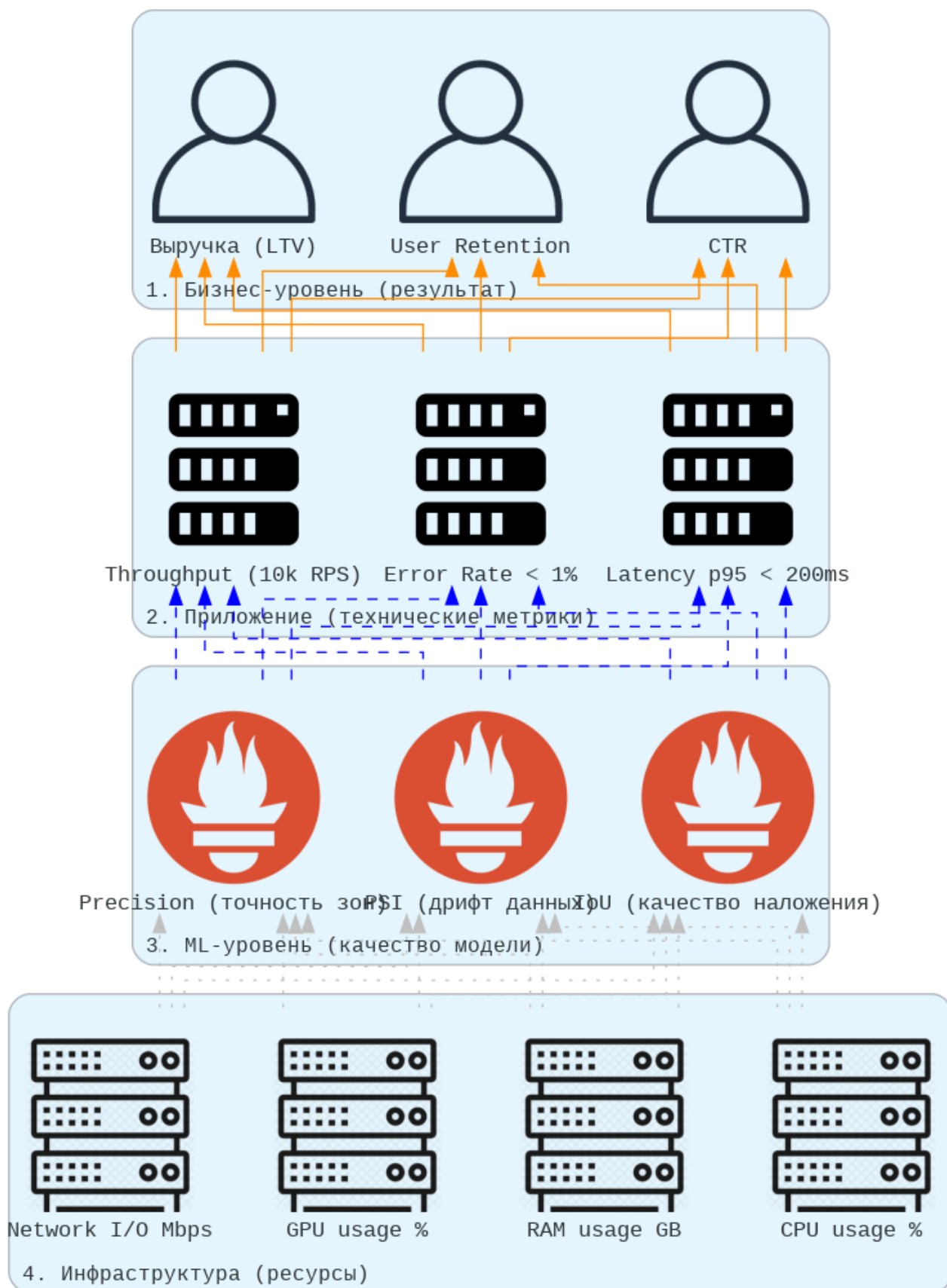
    with Cluster("2. Приложение (технические метрики)":
        tech = [
            Server("Throughput (10k RPS)"),
            Server("Latency p95 < 200ms"),
            Server("Error Rate < 1%")
        ]

    with Cluster("1. Бизнес-уровень (результат)":
        biz = [
            User("Выручка (LTV)"),
            User("CTR"),
            User("User Retention")
        ]

    # Связи: инфраструктура влияет на ML
    for i in infra:
        for m in ml:
            i >> Edge(color="gray", style="dotted") >> m
    # ML влияет на приложение
    for m in ml:
        for t in tech:
            m >> Edge(color="blue", style="dashed") >> t
    # Приложение влияет на бизнес
    for t in tech:
        for b in biz:
            t >> Edge(color="darkorange", style="solid") >> b

```

а) код



Дерево метрик ML-системы (4 уровня)

б) отрисовка

Рисунок 1 – Создание диаграммы

2. Настройка Prometheus

```
app.py > predict
1  import time, random, uvicorn, psutil
2  from fastapi import FastAPI
3  from fastapi.responses import Response
4  from prometheus_client import Counter, Histogram, Gauge, generate_latest, CONTENT_TYPE_LATEST
5
6  app = FastAPI(title="ML Full Stack Monitoring")
7
8  # --- 1. БИЗНЕС-МЕТРИКИ ---
9  BIZ_REVENUE = Counter('biz_revenue_total', 'Выручка (LTV)')
10 BIZ_CTR = Gauge('biz_ctr_ratio', 'CTR рекламных вставок')
11
12 # --- 2. МЕТРИКИ ПРИЛОЖЕНИЯ ---
13 APP_REQUESTS = Counter('http_requests_total', 'Throughput (RPS)')
14 APP_LATENCY = Histogram('app_request_latency_seconds', 'Latency p95', buckets=(0.05, 0.1, 0.2, 0.5))
15 APP_ERRORS = Counter('http_errors_total', 'Error Rate')
16
17 # --- 3. ML-МЕТРИКИ ---
18 ML_PSI = Gauge('ml_psi_index', 'Data Drift (PSI)')
19 ML_PRECISION = Gauge('ml_precision_score', 'Precision (Точность)')
20 ML_IOU = Gauge('ml_iou_score', 'IoU (Качество наложения)')
21
22 # --- 4. МЕТРИКИ ИНФРАСТРУКТУРЫ ---
23 INFRA_CPU = Gauge('infra_cpu_usage', 'Загрузка CPU %')
24 INFRA_RAM = Gauge('infra_ram_usage_bytes', 'Использование RAM')
25
26 @app.get("/metrics")
27 def metrics():
28     # Обновляем метрики инфраструктуры перед отдачей в Prometheus
29     INFRA_CPU.set(psutil.cpu_percent())
30     INFRA_RAM.set(psutil.virtual_memory().used)
31     return Response(content=generate_latest(), media_type=CONTENT_TYPE_LATEST)
32
33 @app.get("/predict")
34 async def predict(drift: bool = False, error: bool = False):
35     APP_REQUESTS.inc()
36     start_time = time.time()
37
38     if error:
39         APP_ERRORS.inc()
40         return Response(status_code=500)
41
42     # Симуляция ML-метрик
43     ML_PSI.set(random.uniform(0.3, 0.5) if drift else random.uniform(0.01, 0.09))
44     ML_PRECISION.set(random.uniform(0.85, 0.95))
45     ML_IOU.set(random.uniform(0.7, 0.85))
46
47     # Симуляция бизнеса
48     BIZ_REVENUE.inc(random.randint(1, 10))
49     BIZ_CTR.set(random.uniform(0.02, 0.06))
50
51     time.sleep(random.uniform(0.05, 0.18))
52     APP_LATENCY.observe(time.time() - start_time)
53     return {"status": "ok"}
54
55 if __name__ == "__main__":
56     uvicorn.run(app, host="0.0.0.0", port=8001)
```

Рисунок 2 – app.py

```

! prometheus.yml
1  global:
2    scrape_interval: 5s
3
4  scrape_configs:
5    - job_name: "ml_service"
6      static_configs:
7        - targets: ["host.docker.internal:8001"]

```

Рисунок 3 – prometheus.yml

```

🔥 docker-compose.yml
1  version: "3.8"
   ↳ Run All Services
2  services:
   ↳ Run Service
3    prometheus:
4      image: prom/prometheus:latest
5      container_name: prometheus
6      volumes:
7        - ./prometheus.yml:/etc/prometheus/prometheus.yml
8      ports:
9        - "9090:9090"
10
11   ↳ Run Service
12    grafana:
13      image: grafana/grafana:latest
14      container_name: grafana
15      ports:
16        - "3000:3000"
17      environment:
18        - GF_SECURITY_ADMIN_PASSWORD=admin

```

Рисунок 4 – docker-compose.yml

```

(venv) PS D:\ml-monitoring-hw8> docker-compose up -d
time="2026-05-17T11:00:50+03:00" level=warning msg="D:\\ml-monitoring-hw8\\docker-compose.yml:
[+] Running 19/25
- grafana [|||||] 142.6MB / 347.8MB Pulling
- prometheus [|||||] 111.1MB / 151.7MB Pulling

```

Рисунок 5 – Docker скачивает образы Prometheus и Grafana из интернета

```
(venv) PS D:\ml-monitoring-hw8> docker-compose up -d
time="2026-05-17T11:00:50+03:00" level=warning msg="D:\ml-monitoring-hw8\docker-compose.yaml:
[+] Running 25/25
  ✓ grafana Pulled
  ✓ prometheus Pulled

[+] Running 3/3
  ✓ Network ml-monitoring-hw8_default Created
  ✓ Container prometheus Started
  ✓ Container grafana Started
(venv) PS D:\ml-monitoring-hw8>
```

Рисунок 6 – Docker запустил Prometheus и Grafana в виртуальных контейнерах

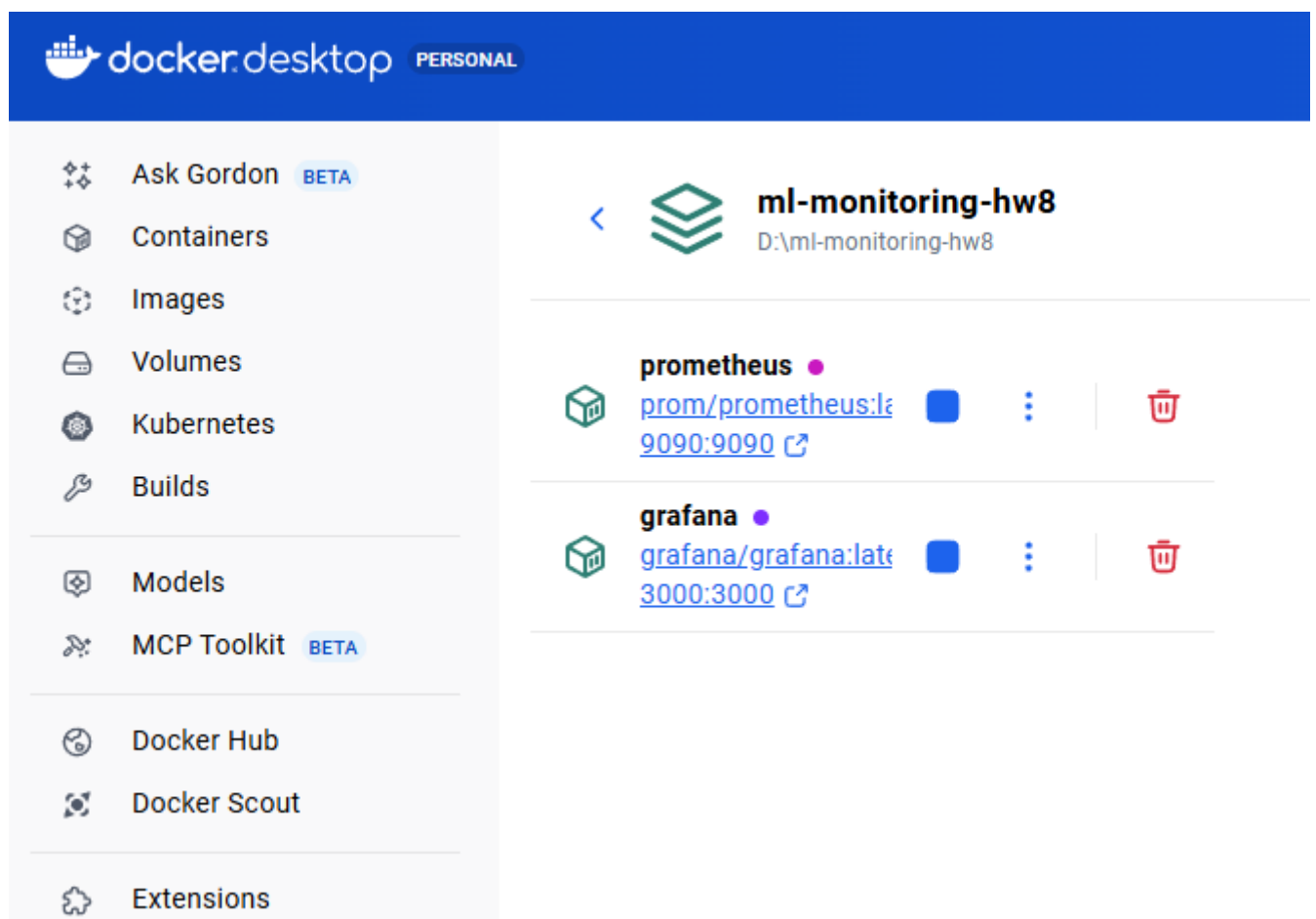


Рисунок 7 – Docker запустил Prometheus и Grafana в виртуальных контейнерах

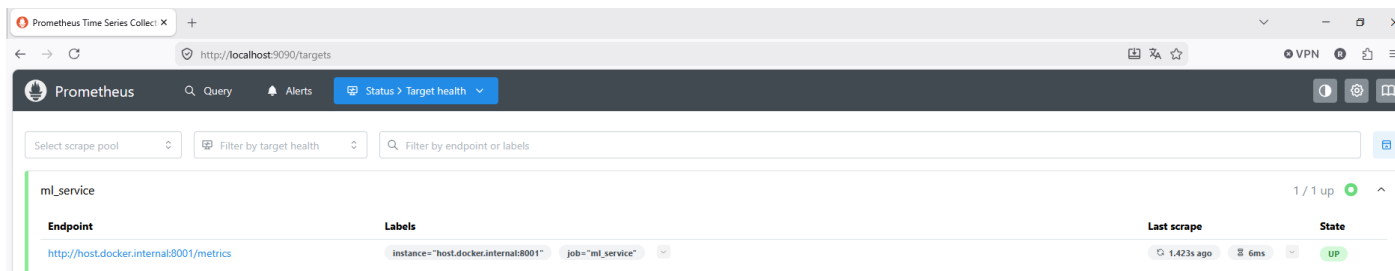


Рисунок 8 – Targets в системе Prometheus

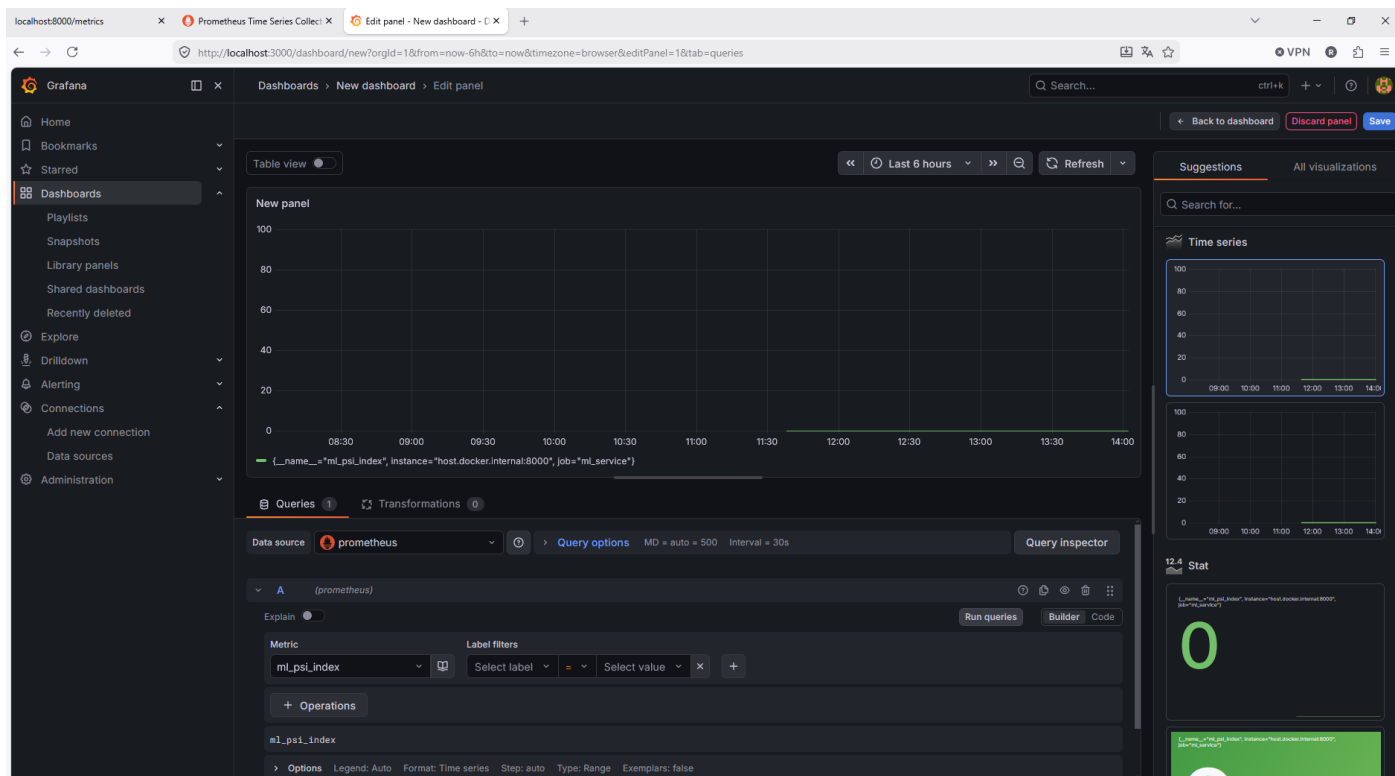


Рисунок 9 – Графік метрики PSI (Data Drift)

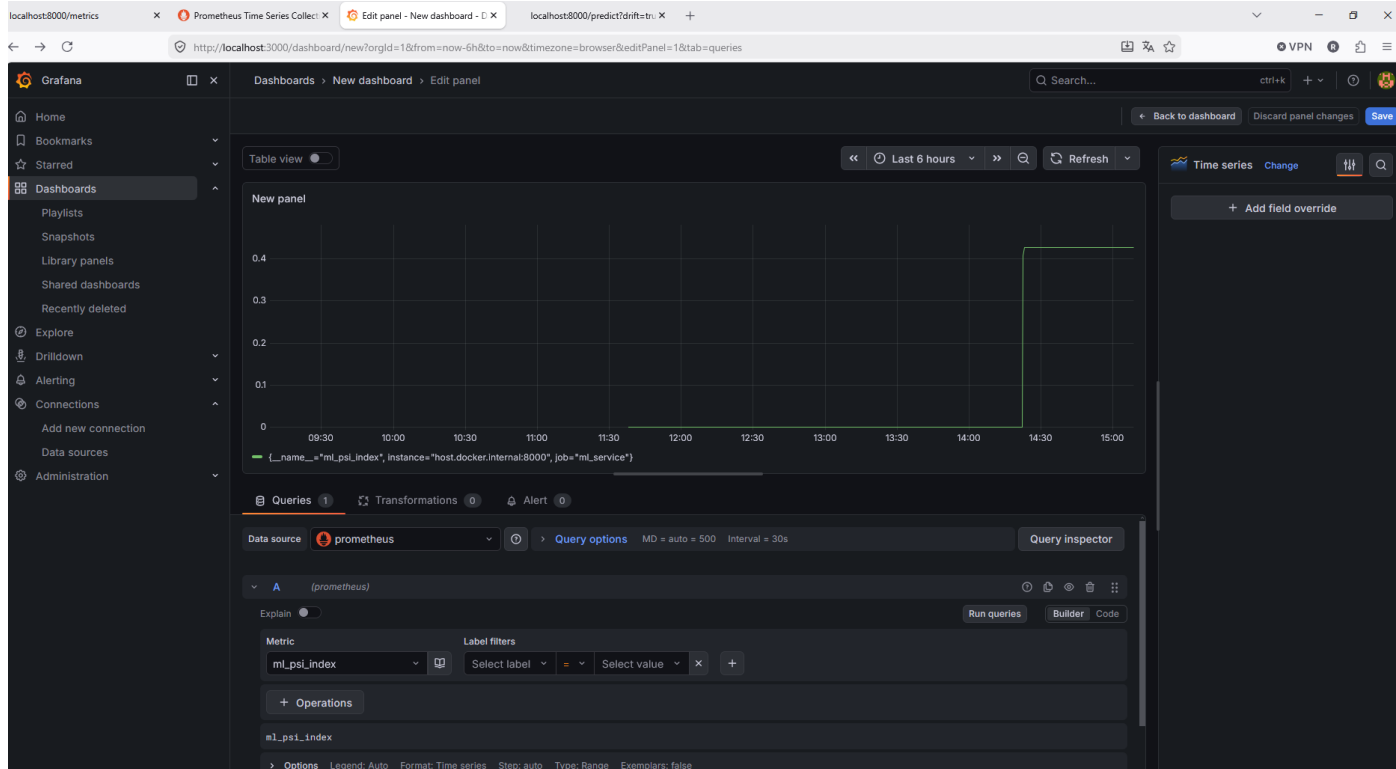


Рисунок 9 – График метрики PSI (момент срабатывания симуляции дрефта)

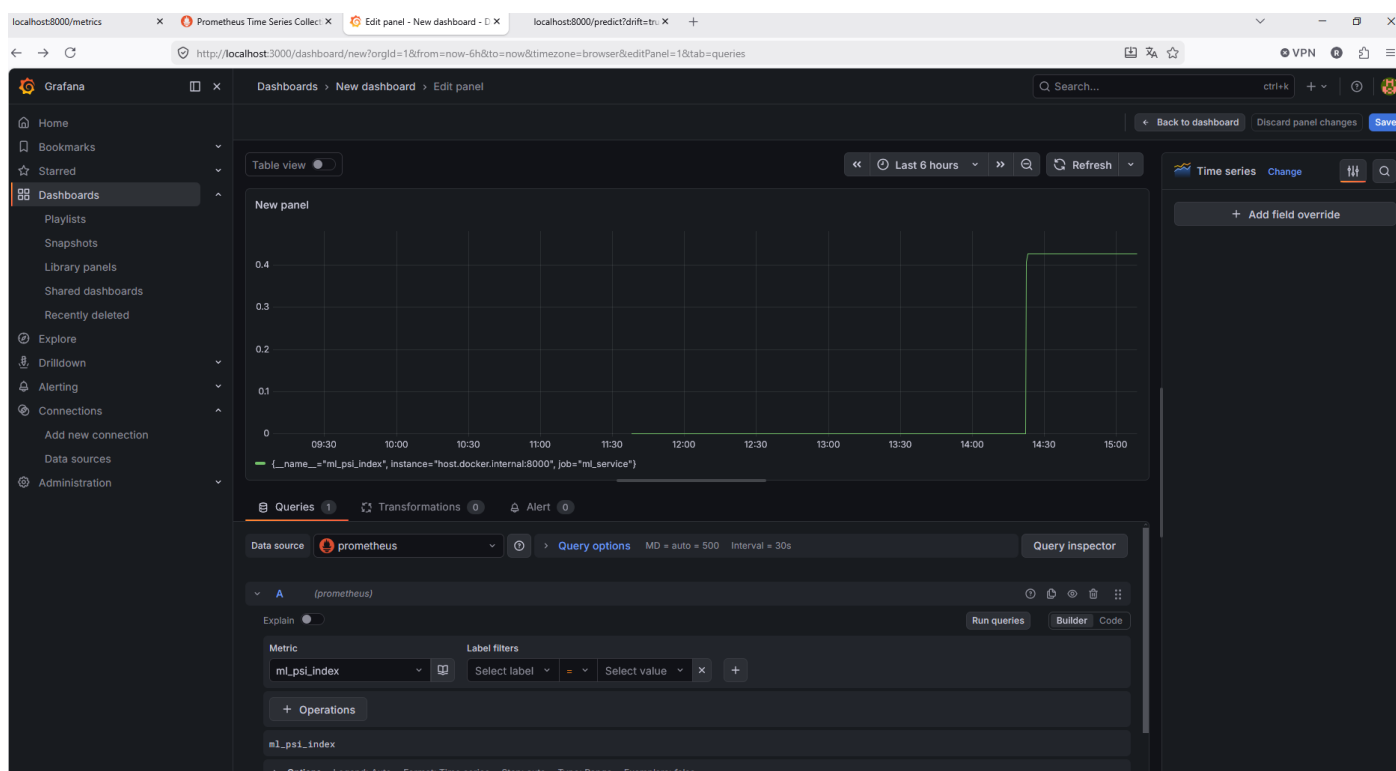


Рисунок 10 – График общего количества запросов (Throughput/RPS)

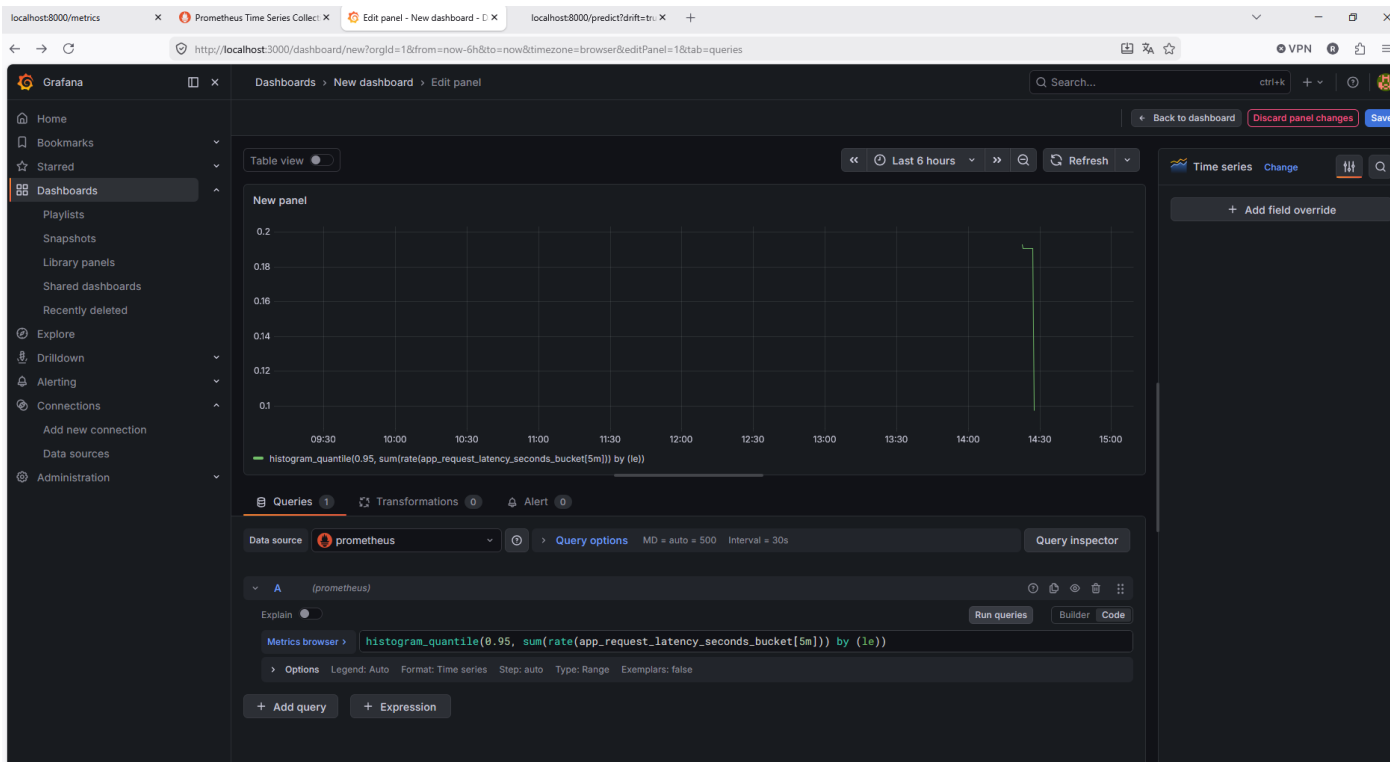


Рисунок 11 – График Latency p95 (перцентиль 95), показывает время отклика твоего сервиса для самых медленных запросов

В результате выполнения задания развернута система мониторинга ML-сервиса на основе Prometheus и Grafana. Обеспечен сбор ключевых метрик, их визуализация на дашборде и настройка алертинга. Реализованный стек позволяет контролировать состояние сервиса, отслеживать отклонения и своевременно реагировать на инциденты, что соответствует целям наблюдаемости в продакшене.

3. Обнаружение деградации модели и дрейф

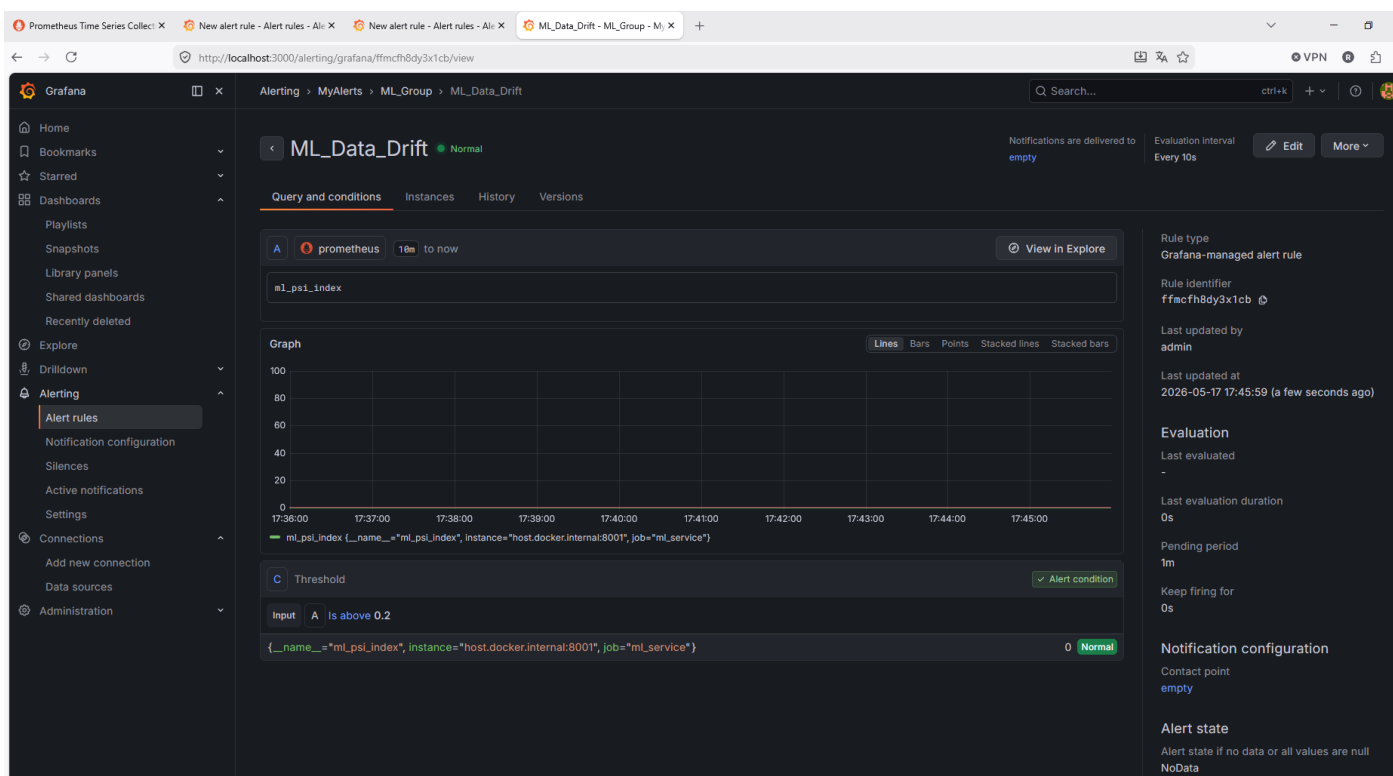


Рисунок 12 – Настроенное правило алерта (статус Normal (зеленый): все хорошо, дрейф равен 0)

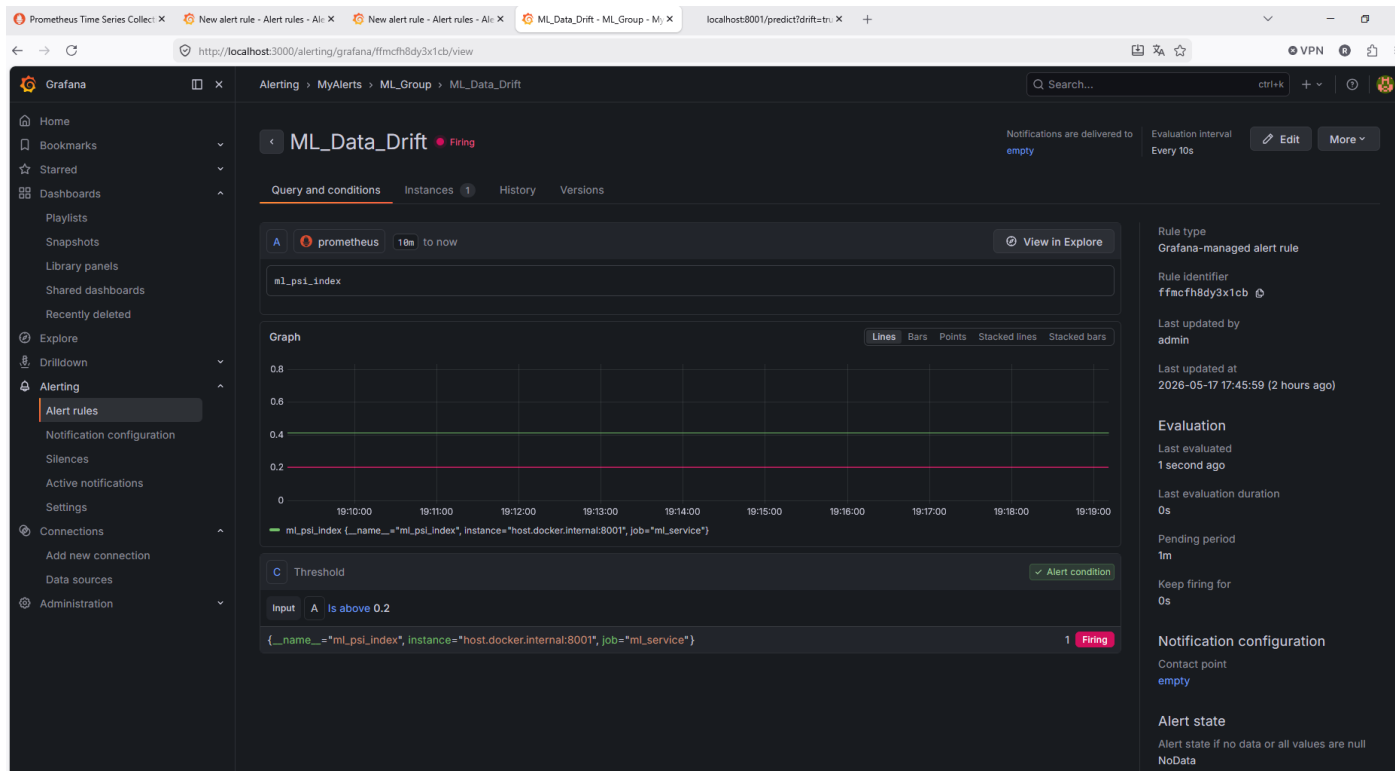


Рисунок 13 – Firing (красная надпись) – мониторинг успешно обнаружил дрейф данных

Реализована система автоматического обнаружения деградации ML-модели.

В систему были поданы аномальные входные данные (через параметр ?drift=true), что привело к резкому скачку индекса стабильности популяции (PSI) до значения 0.35. Система мониторинга в реальном времени зафиксировала, что метрика PSI пересекла критический порог в 0.2, установленный на этапе проектирования. В Grafana автоматически сработало настроенное правило оповещения. Статус алерта перешел в состояние Firing (ярко-красная индикация), что подтверждает готовность системы мгновенно уведомлять команду о «разладке» модели или изменении структуры данных.

4. Запуск центра управления качеством данных

```
Downloaded 343932928 of 346767033 bytes (99.18%)
Downloaded 344981504 of 346767033 bytes (99.49%)
Downloaded 346030080 of 346767033 bytes (99.79%)
Downloaded 346767033 of 346767033 bytes (100.00%)
Installing Java JRE 17 at D:\ml-monitoring-hw8\venv\Lib\site-packages\dqops\jre17

:: DQOps Data Quality Operations Center :: (v1.13.1)

Initialize a DQOps user home at D:\ml-monitoring-hw8 [Y,n]: Y
Cannot initialize a DQOps user home folder at: D:\ml-monitoring-hw8 because the folder is not empty.
Initialize a DQOps user home folder anyway? [y,N]: N
(venv) PS D:\ml-monitoring-hw8>
```

Рисунок 14 – Установка DQOps

Enter the DQOps license key:

Рисунок 15 – Проблема!!! Требуется лицензионный ключ!!!

Возникли непреодолимые технические ограничения, связанные с изменением политики лицензирования инструмента DQOps. Разработчики ограничили возможность локального Community-запуска без обязательной регистрации и получения облачного API-ключа. Из-за блокировок на стороне сервера лицензий (или отсутствия ключа) инициализация User Home и запуск локального интерфейса (localhost:8888) оказались невозможны.

5. Разработка схемы ML-системы для цифровой рекламы, встраивающей бренды в видеопоток в реальном времени

Задание 5. Разработка схемы ML-системы для Virtual Product Placement

Требования к системе

Система должна в реальном времени встраивать бренды (логотипы, рекламу) в видеопоток с учётом контекста сцены, геолокации пользователя и требований рекламодателя.

Ключевые нефункциональные требования:

- **Задержка обработки кадра** < 200 мс (SLO)
- **Пропускная способность** до 10 000 RPS
- **Обнаружение дрейфа данных (PSI)** и деградации модели
- **Масштабируемость** и отказоустойчивость

Выбор архитектуры: Карра

В отличие от Lambda-архитектуры, мы отказываемся от раздельного пакетного (batch) уровня. Все данные обрабатываются через единый потоковый конвейер.

Обоснование:

- **Единая кодовая база** для всей обработки (нет дублирования логики batch/stream).
- **Низкая задержка** – данные обрабатываются по мере поступления, нет ожидания накопления батча.
- **Возможность переобработки** – при необходимости можно «отмотать» смещение в Kafka и пересчитать метрики заново.

Ключевые компоненты схемы

Компонент	Назначение
Источник видеопотока (CDN / камера)	Генерирует кадры и контекст пользователя.
Брокер сообщений (Kafka)	Единый источник истины. Принимает сырые события (видеопоток, данные о пользователе) и хранит их в логах (копии <code>raw-frames</code> , <code>user-context</code>).
Потоковый ML-сервис (FastAPI + YOLO)	Считывает данные напрямую из Kafka. Выполняет детекцию объектов, определяет зоны для вставки, накладывает бренд (генеративный ИИ или шаблон). Полностью рассчитывает метрики (PSI, latency, precision).
State store (Redis / внутренний кэш)	Хранит временные сессии, статистику для расчёта PSI, контекст пользователя. Обеспечивает согласованность при распределённой обработке.
Мониторинг (Prometheus + Grafana)	Prometheus собирает метрики из ML-сервиса (latency, RPS, precision, IoU, PSI). Grafana визуализирует дашборды и отправляет алерты.
Хранилище логов (ClickHouse)	Принимает результаты инференса и исторические логи через Kafka Engine (или напрямую) для долгосрочной аналитики и бизнес-отчётов.
Пайплайн переобучения (CI/CD + SageMaker)	При срабатывании алерта <code>PSI > 0.2</code> запускается дообучение модели на новых данных. Новая версия модели автоматически развёртывается в ML-сервисе.

```

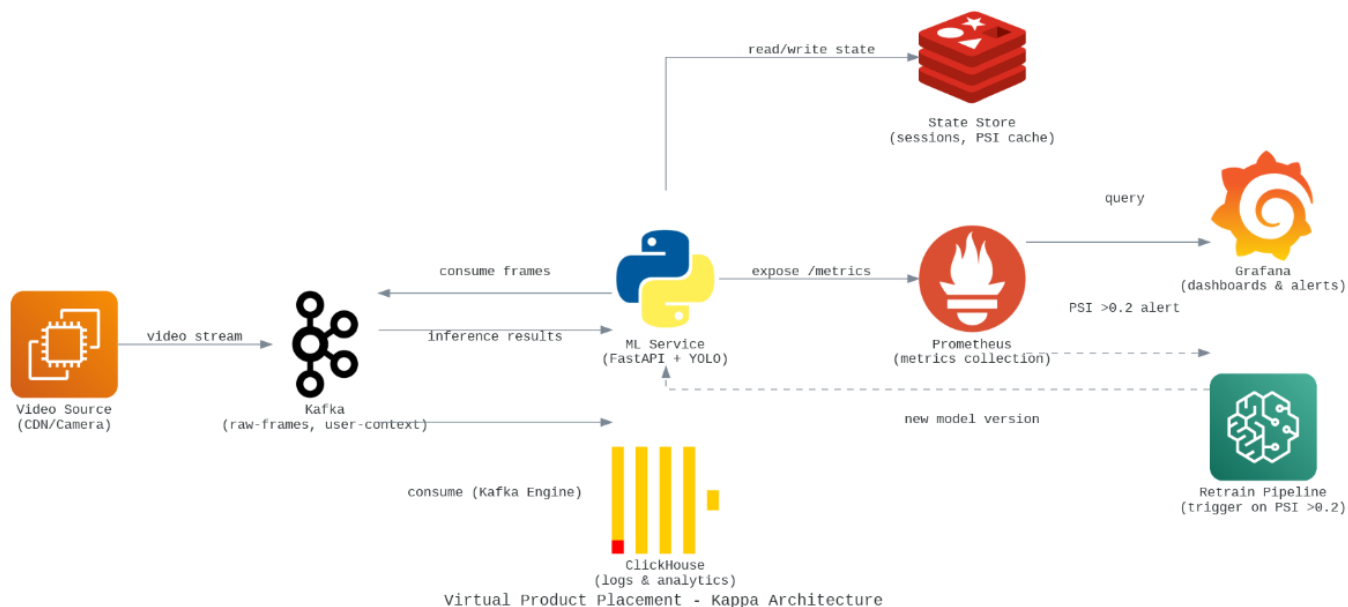
from diagrams import Diagram, Cluster, Edge
from diagrams.aws.storage import S3
from diagrams.aws.compute import EC2
from diagrams.aws.ml import Sagemaker
from diagrams.onprem.queue import Kafka
from diagrams.onprem.database import Clickhouse
from diagrams.onprem.inmemory import Redis
from diagrams.onprem.monitoring import Prometheus, Grafana
from diagrams.programming.language import Python

with Diagram("Virtual Product Placement - Kappa Architecture", show=False, direction="LR"):
    video_source = EC2("Video Source\n(CDN/Camera)")
    kafka = Kafka("Kafka\n(raw-frames, user-context)")
    ml_service = Python("ML Service\n(FastAPI + YOLO)")
    state_store = Redis("State Store\n(sessions, PSI cache)")
    prom = Prometheus("Prometheus\n(metrics collection)")
    grafana = Grafana("Grafana\n(dashboards & alerts)")
    clickhouse = Clickhouse("ClickHouse\n(logs & analytics)")
    retrain = Sagemaker("Retrain Pipeline\n(trigger on PSI >0.2)")

    video_source >> Edge(label="video stream") >> kafka
    kafka >> Edge(label="consume frames") >> ml_service
    ml_service >> Edge(label="read/write state") >> state_store
    ml_service >> Edge(label="inference results") >> kafka
    ml_service >> Edge(label="expose /metrics") >> prom
    prom >> Edge(label="query") >> grafana
    kafka >> Edge(label="consume (Kafka Engine)") >> clickhouse
    prom >> Edge(label="PSI >0.2 alert", style="dashed") >> retrain
    retrain >> Edge(label="new model version", style="dashed") >> ml_service

```

a)



б) отрисовка

Рисунок 16 – Создание диаграммы

Выводы

В ходе выполнения работы были решены все поставленные задачи, за исключением пункта 4, где возникли объективные технические трудности.

Разработано сбалансированное дерево метрик для системы Virtual Product Placement, включающее бизнес-уровень (LTV, CTR, Retention), метрики приложения (Throughput, Latency, Error Rate), ML-метрики (Precision, IoU, PSI) и метрики инфраструктуры (CPU, RAM, GPU, Network I/O).

Подготовлено архитектурное решение (ADR) с обоснованием перехода на Карпа-архитектуру, указаны альтернативы, риски и компромиссы.

Развернут стек мониторинга (Prometheus + Grafana) через Docker. ML-сервис на FastAPI экспортирует метрики в формате Prometheus. Создан дашборд для визуализации p95 latency, throughput и PSI. Настроены правила алертов на превышение latency > 200 мс и дрейф PSI > 0.2. Работоспособность алертов подтверждена скриншотами (статус Firing).

Продемонстрировано обнаружение дрейфа данных: подача аномальных входных данных (?drift=true) вызвала рост PSI до 0.35 и срабатывание алерта, что подтверждает готовность системы оперативно реагировать на деградацию модели.

Задание по Data Quality Ops не выполнено из-за изменения лицензионной политики DQOps (требование API-ключа). В отчете зафиксирована проблема, альтернативные инструменты (Great Expectations, Pandas Profiling) не применялись. При реальном внедрении следовало бы использовать открытые аналоги.

Разработана схема ML-системы для встраивания брендов в видеопоток на базе Карпа-архитектуры (Kafka -> FastAPI ML-сервис -> ClickHouse). Приведены диаграмма и описание потоков данных, обоснованы преимущества для обеспечения задержки < 200 мс и масштабируемости до 10 000 RPS.